

Secure IoT Communication Using mbedTLS on ESP32-S3

Michael Woldai
dept. of Electrical and Computer
Engineering
California State University Chico
Chico, CA, United States
mwoldai@csuchico.edu

Cater James Trudeau
dept. of Electrical and Computer
Engineering
California State University Chico
Chico, CA, United States
ctrudeau@csuchico.edu

Abstract— This project demonstrates a secure device-to-device communication system implemented on ESP32-S3 microcontrollers using the mbedTLS cryptographic library. A custom secure channel is constructed over Bluetooth Low Energy (BLE) using Elliptic Curve Diffie–Hellman (ECDH) key exchange on the secp256r1 curve to establish a shared session secret. The shared secret is converted into a fixed-length symmetric key using SHA-256 and used for AES-256 encryption of transmitted data. The system protects the confidentiality of simulated sensor messages and illustrates secure computation concepts including asymmetric key exchange, symmetric encryption, key derivation, and forward secrecy on resource-constrained embedded hardware. Practical implementation challenges, including key derivation of consistency and padding errors, are analyzed and resolved.

Keywords— Secure Computation, IoT Security, ESP32-S3, mbedTLS, ECDH, AES-256, BLE

I. INTRODUCTION AND MOTIVATION

The rapid growth of Internet of Things (IoT) devices in health, environmental monitoring, and wearable applications has increased the importance of secure data transmission. Wireless sensor data transmitted without encryption is vulnerable to eavesdropping, manipulation, and replay attacks. This project is motivated by the need to ensure confidentiality and integrity of sensor data in low-power embedded systems.

This work directly supports the development of a wearable air-quality monitor capstone project by adding a cryptographically secure communication layer. The project applies secure computation principles taught in EECE 644 by implementing a complete encrypted

communication pipeline on embedded hardware rather than relying on abstract simulations.

The mbedTLS library was selected due to its widespread industry adoption and integration with the ESP32 platform. By leveraging established cryptographic primitives rather than custom implementations, the project reflects best practices used in commercial embedded security solutions.

II. SYSTEM ARCHITECTURE AND DESIGN

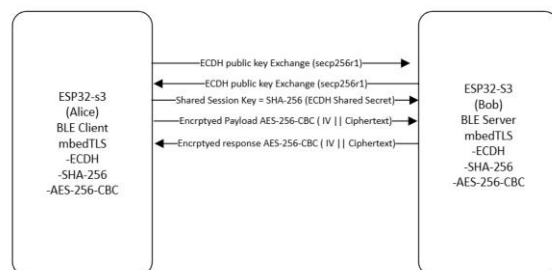


Fig1. Functional block diagram for project.

The implemented system consists of two ESP32-S3 microcontrollers communicating over Bluetooth Low Energy using a custom secure protocol. One ESP32 operates as a BLE server, advertising a GATT service and characteristics, while the second ESP32 functions as a BLE client.

Fig. 1 illustrates the high-level system architecture used in this project. The system is modeled using the classical Alice–Bob abstraction common from class lectures in secure computation EECE 644. Alice represents the ESP32-S3 BLE

client, while Bob represents the ESP32-S3 BLE server. Both devices utilize the mbedTLS library to perform cryptographic operations.

Upon connection, the devices exchange elliptic-curve public keys and perform an ECDH key exchange using the secp256r1 curve. This exchange produces a shared secret without transmitting private keys. The shared secret is then transformed into a fixed-length session key using SHA-256. All subsequent messages are encrypted using AES-256 in CBC mode and transmitted through BLE characteristic writes and notifications.

Fig. 1. Secure BLE communication architecture between two ESP32-S3 devices (Alice and Bob). The devices perform an ECDH key exchange to derive a shared session key, which is then used for AES-256-CBC encrypted message exchange.

III. CRYPTOGRAPHIC FOUNDATIONS

The security of the system is based on a combination of asymmetric and symmetric cryptographic techniques. ECDH enables two devices to derive a common secret over an insecure channel while providing forward secrecy through ephemeral key generation. Even if a device's long-term keys are compromised, previously captured traffic remains protected.

Once the shared secret is generated, symmetric encryption is used for efficiency. AES-256 provides strong confidentiality with relatively low computational overhead, making it suitable for embedded platforms. SHA-256 is used as a key derivation function to convert the elliptic-curve shared secret into a fixed-length symmetric key suitable for AES.

The ESP32-S3 platform includes hardware acceleration for AES and SHA operations, which mbedTLS can utilize through the ESP-IDF and Arduino frameworks, reducing CPU load and power consumption.

IV. IMPLEMENTATION DETAILS

The system was implemented in C++ using the Arduino framework with direct access to mbedTLS cryptographic primitives. BLE functionality was implemented using the ESP32 BLE stack, including GATT services, characteristics, and notifications.

During connection setup, the BLE server generates an elliptic-curve key pair and exposes its public key through a readable characteristic. The client reads this value, generates its own key pair, and sends its public key back to the server. Both devices independently compute the ECDH shared secret and derive an AES session key.

Messages entered through the serial interface are encrypted using AES-256-CBC with a randomly generated initialization vector (IV). The IV is prepended to the ciphertext and transmitted to the receiving device, which decrypts the message using the derived session key.

V. DEBUGGING AND CHALLENGES

During testing, an issue was encountered where AES decryption failed due to padding errors despite successful ECDH handshakes. Debugging revealed that the ECDH shared secret was initially exported using a variable-length representation before hashing. Because elliptic-curve shared secrets may contain leading zero bytes, hashing a variable-length buffer caused the client and server to derive different AES keys.

This issue was resolved by exporting the ECDH shared secret as a fixed 32-byte value, corresponding to the size of the secp256r1 curve, before applying SHA-256. This ensured consistent key derivation on both devices and eliminated decryption and padding errors. This debugging process highlighted the importance of canonical key representations in cryptographic implementations.

VI. RESULTS AND TESTING

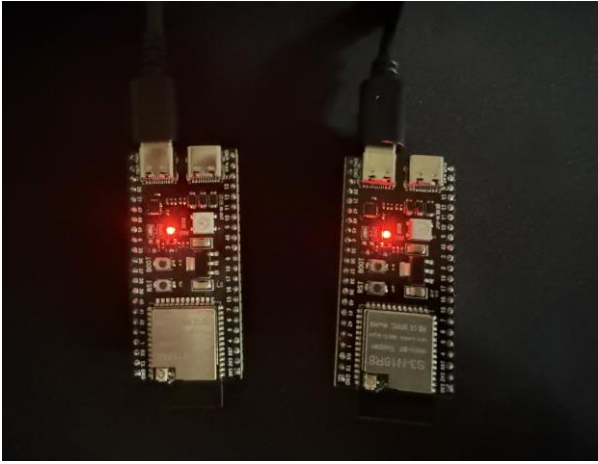


Fig 2. ESP32-S3 boards setup during testing.

The completed system successfully establishes a secure Bluetooth Low Energy (BLE) connection between two ESP32-S3 microcontrollers and exchanges encrypted messages in both directions. The system was evaluated using live hardware tests with both devices operating concurrently.

Fig. 2 shows the physical hardware setup used during testing. Two ESP32-S3 development boards were connected over BLE and demonstrated successful encrypted message exchange. Serial monitor output confirms completion of the ECDH handshake, correct session key derivation, and successful AES-256 decryption of transmitted messages.

Serial monitor output confirms correct protocol operation, including successful ECDH key exchange, consistent session key derivation, and correct decryption of transmitted ciphertext. Encrypted messages differed for each transmission due to the use of randomized initialization vectors, while decrypted plaintext matched the original input exactly. Shown in Appendix A.

Testing confirmed the following behaviors:

- Successful ECDH key exchange and session establishment on each connection

- Correct derivation of identical AES-256 session keys on both devices
- Reliable encryption and decryption of bidirectional messages over BLE
- Proper handling of reconnections with fresh key exchanges

These results verify that the implemented system provides a functional and secure communication channel suitable for embedded IoT applications.

Fig. 2. ESP32-S3 development boards used during testing of the secure BLE communication system.

VII. SECURITY DISCUSSION

The implemented design ensures confidentiality of transmitted data and provides forward secrecy through ephemeral ECDH key exchange. While AES-CBC provides strong encryption, it does not include built-in authentication. Future enhancements could incorporate authenticated encryption modes such as AES-GCM or message authentication codes to provide integrity and tamper with detection.

Despite these limitations, the system demonstrates a secure communication foundation learned from class and suitable for embedded IoT devices operating under resource constraints.

VIII. CONCLUSION AND FUTURE WORK

This project successfully applies secure computation concepts learned from EECE 644 to a real embedded system. By implementing ECDH-based key exchange and AES-256 encryption on ESP32-S3 hardware, the project demonstrates how cryptographic theory can be translated into practical IoT security solutions.

The final system establishes a secure BLE communication channel between two embedded devices, supports bidirectional encrypted messaging, and handles reconnections with fresh

session keys. Debugging and validation efforts reinforced the importance of canonical key derivation and careful cryptographic implementation in resource-constrained environments.

Future work may include the integration of authenticated encryption modes such as AES-GCM, certificate-based authentication, and direct integration with sensor hardware for encryption in wearable applications.

REFERENCES

- [1] ARM Ltd., “*mbedTLS v3.6.0 Documentation*,” 2025. [Online]. Available: <https://mbed-tls.readthedocs.io/projects/api/en/v3.6.4/>
- [2] Espressif Systems, “*ESP32-S3 Datasheet and ESP-IDF Programming Guide*,” 2025. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>
- [3] STMicroelectronics, “*STSAFE-A110: Authentication and Secure Communication for IoT Devices*,” 2025. [Online]. Available: https://www.st.com/content/st_com/en/campaigns/stsafel010-authentication-consumables-peripherals-z24.html
- [4] S. Mukherjee, A. Dutta, and R. Singh, “*Lightweight TLS for IoT Devices*,” *IEEE Internet of Things Journal*, 2022.
- [5] Z. Wang, *EECE 644 Class Notes: Secure Computation and Cryptographic Protocols*, California State University, Chico, 2025.

Appendix A: Serial Log Evidence

This appendix provides Client and Server serial monitor output collected during live testing of the secure BLE communication system. The logs demonstrate successful ECDH key exchange, AES session key derivation, encrypted message transmission, and correct decryption on both devices.

```
Decrypted text: War begin when you will, but they do not end when you please -Machiavelli
```